DAY-21

PYTHON 90 DAYS CHALLENGE

Chapter 12. Scope and Namespaces in Python

Scope means the area of a program where a variable can be accessed.

Namespace means a place where names are stored and mapped to objects.

name -> object/value

Example: x = 10

Output: Aman

Invalid outside access:

```
PYTHON CODE
print(name)
```

Output: NameError: name 'name' is not defined.

12.2 Global Scope

A **global scope** is the main area of the program. Variables created outside all functions are global variables.

Syntax

NOTE: Global variables are accessible throughout the file after they are defined.

12.3 Local vs Global Scope

Point	Local Scope	Global Scope
Created where?	Inside function	Outside functions
Accessible where?	Only inside that function	Inside and outside functions

n

Output:

Local Aman
Global Aman

NOTE: Inside the function, the local variable gets priority over the global variable.

12.5 global Keyword

The global keyword is used to modify a global variable inside a function.

Syntax

global variable_name

```
increase_count()
```

Output: UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

NOTE: Use global only when you really need to modify a global variable. Better style is usually to return a value:

PYTHON CODE

```
def increase_count(count):
    return count + 1

count = 0
count = increase_count(count)

print(count)
```

```
        print(message)

    inner()
outer()
```

Output: Hello

12.7 nonlocal Keyword

The nonlocal keyword is used to modify a variable from the nearest enclosing function scope.

Syntax

nonlocal variable_name

```
    inner()

outer()
```

Output: UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

global vs nonlocal

Keyword	Used For	Scope Affected
global	Modify global variable	Global scope
nonlocal	Modify enclosing function variable	Enclosing scope

LEGB is the order Python follows while searching for a variable name.
L -> Local
E -> Enclosing
G -> Global
B -> Built-in

When Python sees a variable name, it searches in this order:

1. **Local scope**: inside the current function.
2. **Enclosing scope**: inside outer functions.
3. **Global scope**: main program/file.
4. **Built-in scope**: built-in Python names.

Example 1

1. A namespace maps names to values or objects.
2. Python uses namespaces to avoid name conflicts.
3. Different scopes have different namespaces.
4. Local, global, and built-in scopes each have their own namespaces.
5. The same name can exist in different namespaces without conflict.

Example 1

PYTHON CODE

```
x = 100
def show():
    x = 50
    print(x)

show()
```

1. locals() returns the current local namespace as a dictionary.
2. globals() returns the global namespace as a dictionary.
3. These are mainly used for debugging and learning.
4. Usually, we should not modify program logic using them.

Example 1

PYTHON CODE

```
course = "Python"

def show():
    name = "Aman"
    print(locals())
```

Error	Meaning
NameError	Name not found in any scope
UnboundLocalError	Local variable used before assignment

12.13 Important Scope Summary Table

Concept	Meaning
Local scope	Variable inside current function
Global scope	Variable outside all functions